Introducción a Visual Studio

Antonio Espín Herranz

Contenidos

- Visión general del IDE
- Parecidos y diferencias con Visual Studio Code
- Entorno del curso: Windows + Visual Studio + vcpkg + Docker Desktop.

Visión general del IDE

- Visual Studio está pensado para:
  - **Proyectos grandes y complejos**
  - **Desarrollo profesional en Windows**
  - **Integración profunda con MSVC**
  - **Depuración avanzada**
  - **Entornos con múltiples módulos, librerías y dependencias**
  - **Integración con herramientas de Microsoft (Docker, Azure, Windows SDK)**
- Es un IDE orientado a **productividad, estabilidad y herramientas integradas**, no a ligereza o minimalismo.

- ***Más info en la comparativa con VSCode***

VS vs VSCode

Aspecto	Visual Studio 2022	Visual Studio Code
Tipo	IDE completo	Editor extensible
Compilador	MSVC integrado	Usa MSVC, GCC o Clang según configuración
CMake	Integración nativa	Extensión CMake Tools
vcpkg	Integración automática	Requiere configuración manual
Depuración C++	Muy avanzada	Buena, pero depende de extensiones
Docker	Integración directa	Extensiones, más manual
gRPC	Mejor con MSVC	Más natural en contenedores Linux
Crow	Funciona perfecto	Funciona perfecto
WebSockets	Depuración más cómoda	Más flexible en contenedores
Peso	Pesado	Ligero
Uso ideal	Windows puro, MSVC, proyectos grandes	Multiplataforma, contenedores, setups híbridos

Elegir Visual Studio

- **1. Trabajas con MSVC como compilador principal**
  - Integración profunda con el toolchain de Microsoft.
  - Depuración de C++ extremadamente sólida.
  - Análisis estático y dinámico integrados.

- **2. Usas CMake + vcpkg en Windows**
  - Detecta automáticamente el toolchain de vcpkg.
  - No necesitas configurar rutas manuales.
  - CMake Presets funcionan de forma transparente.

- **3. Necesitas depurar Crow, gRPC o WebSockets en Windows**
  - Breakpoints estables incluso en callbacks asíncronos.
  - Inspección de memoria y buffers binarios (protobuf).
  - Análisis de rendimiento integrado.

- **4. Quieres integrar Docker Desktop sin complicarte**
  - Extensión oficial de Docker.
  - Puedes depurar dentro del contenedor Windows.
  - Generación automática de Dockerfile.

- **5. Proyectos grandes, enterprise o con muchos módulos**
  - Solution Explorer escala mejor.
  - Mejor gestión de configuraciones complejas.

Elegir Visual Studio

- **1. Trabajas con contenedores Linux**
  - Dev Containers es una maravilla para C++ moderno.
  - Puedes compilar con GCC/Clang dentro del contenedor.
  - El entorno es 100% reproducible para alumnos/equipos.
- **2. Quieres un entorno ligero y multiplataforma**
  - Funciona igual en Windows, Linux y macOS.
  - Ideal para microservicios que se despliegan en Linux.
- **3. Necesitas flexibilidad extrema**
  - Puedes mezclar toolchains: MSVC, GCC, Clang.
  - Puedes trabajar directamente dentro de Docker.
  - Extensiones para gRPC, protobuf, CMake, etc.
- **4. Enseñanza o demos rápidas**
  - Arranca en segundos.
  - Configuración mínima.
  - Perfecto para mostrar conceptos sin ruido.

Comparativa por Herramienta

- **1. CMake**
  - **Visual Studio**
    - Soporte nativo.
    - CMake Presets integrados.
    - Vista de targets muy clara.
  - **VS Code**
    - Extensión CMake Tools.
    - Más flexible para entornos Linux.
    - Requiere configurar kits y toolchains.
- **2. vcpkg**
  - **Visual Studio**
    - Integración automática.
    - No necesitas configurar rutas.
    - Detecta triplets y toolchain sin esfuerzo.
  - **VS Code**
    - Necesitas configurar:
      - CMAKE_TOOLCHAIN_FILE
      - VCPKG_ROOT
      - Kits de CMake
    - Más trabajo, pero más control.

Herramientas

- **Visual Studio Comunity**: https://visualstudio.microsoft.com/es/downloads
- Descarga un instalador, seleccionar desarrollo en C++
- En los detalles de la instalación tener en cuenta la versión de Windows 10 u 11

- **Cmake**: https://cmake.org/download/
  - Se puede elegir la distribución msi o zip.
  - Añadir al **PATH**.

- **Git**: https://git-scm.com/downloads
  - Instalar con las opciones por defecto

Comparativa por Herramienta

- **3. Depuración C++**
  - **Visual Studio**
    - El mejor depurador C++ en Windows.
    - Inspección profunda de memoria.
    - Breakpoints en callbacks asíncronos (Crow, gRPC).
    - Visualizadores de tipos personalizados.
- **VS Code**
  - Depuración buena, pero:
    - Depende de extensiones.
    - Menos estable con MSVC.
    - Mucho mejor con GCC/Clang en Linux.
- **4. Docker Desktop**
  - **Visual Studio**
    - Integración directa.
    - Depuración dentro del contenedor Windows.
    - Generación automática de Dockerfile.
  - **VS Code**
    - Extensiones para Docker.
    - Dev Containers → experiencia superior en Linux.
    - Ideal para reproducibilidad en equipos.

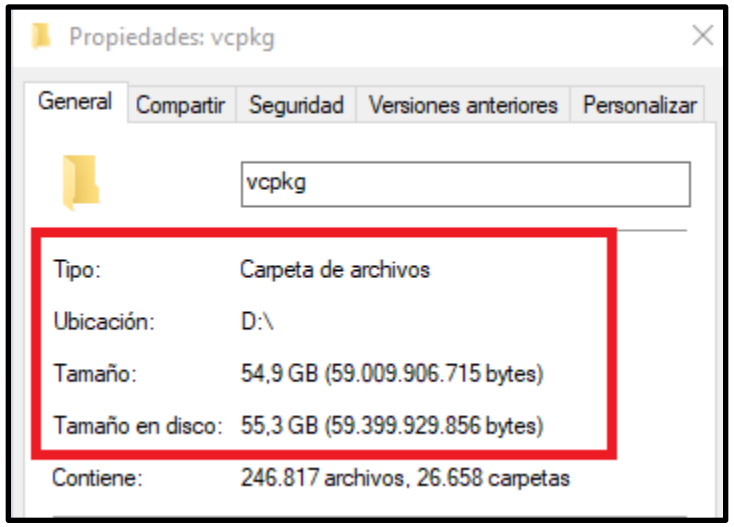
Herramientas

- También se puede instalar **GitHub Desktop**
- https://www.gitkraken.com/download

- **vcpkg**: gestor de paquetes y librerías de C++
- Hay que clonar un repositorio de Git
- ***git clone https://github.com/microsoft/vcpkg.git***
- ***cd vcpkg***
- ***.\bootstrap-vcpkg.bat***

- **Al ejecutar el bat ya veremos el .exe**
- **Añadir al PATH (la carpeta vcpkg)**

OJO con el espacio en disco



Luego se puede liberar espacio

vcpkg

- Una vez instalado y añadido al **PATH** ya se puede utilizar para instalar librerías.
  - **OJO, hay algunas librerías que tardan poco, pero otras pueden tardar 15 o 20 min.**

- El formato para instalar las librerías (en una consola):
  - **vcpkg install librería**

- Se pueden instalar varias a la vez (***pero mejor por separado por si falla alguna***):
  - **vcpkg install librería1 librería2 ...**

- Una vez instaladas, se **integran en Visual Studio** con:
  - **vcpkg integrate install**

Instalar librerías (ejemplo con crow)

- Ir a la carpeta de **vcpkg** (***si está en el Path no es necesario***)
- Para instalar:
- **vcpkg install crow**

- Para integrar con Visual Studio (***no hay que copiar los archivos***)
- **vcpkg integrate install**

- Debería dejarnos en un proyecto de Visual Studio:
- **#include <crow.h>**

vcpkg

- **Problemas con la instalación de librerías**, hay veces que al instalar alguna librería podemos tener algún problema, en estos casos:
  - Ir a la carpeta donde tengamos el repositorio de **vcpkg**
  - Actualizar el repositorio → **git pull**
  - Actualizar el ejecutable de vcpkg (ejecutar el bat) → **bootstrap-vcpkg.bat**
  - Actualizar paquetes disponibles → **vcpkg update**
  - Reinstalar una librería que falló en la instalación →
    - **vcpkg remove librería**
    - **vcpkg install librería**

vcpkg instalar librerías

- Desde una consola, lanzar los comandos:

  vcpkg install crow

  vcpkg install grpc

  … …

Lista completa de librerías
En las siguientes páginas

- Con esta instrucción se integran las librerías en Visual Studio
  - Se puede ejecutar después de instalar cada librería o al final.

    vcpkg integrate install

vcpkg: librerías a instalar | **vcpkg install nombre_libreria**

- Protocolos HTTP y webSockets → **boost-beast**
- APIs RESTful → **crow**
- Para json → **nlohmann-json**

vcpkg: librerías a instalar II
vcpkg install nombre_libreria

- Comunicaciones → **gprc** y **protobuf**
  - Protobuf lo necesita grpc
  - Con protobuf se instala un comando llamado **protoc** y hay que añadirlo al **PATH**

  - Suele estar en esta ruta:
  - C:\vcpkg**installed\x64-windows\tools\protobuf\protoc.exe**

  - Comprobar si se ha instalado el plugin en esta ubicación:
  - C:\vcpkg**installed\x64-windows\tools\grpc\grpc_cpp_plugin.exe**

  - **Si no vemos el plugin, lanzar estos comandos:**
    - **vcpkg remove grpc**
    - **vcpkg install grpc --recurse**
    - **vcpkg integrate install**

vcpkg: librerías a instalar III
vcpkg install nombre_libreria

- Base de datos MySQL → **libmysql**
- Base de datos PostGreSQL → **libpq**

- **SOLO Si da problemas la librería de mysql: libmysql**
- Ir a la carpeta de vcpkg
  - git pull
  - vcpkg update
  - vcpkg upgrade
    - Puede dar un warning y habrá que ejecutar con:
    - vcpkg upgrade --no-dry-run

- Después de actualizar:
  - vcpkg remove libmysql
  - vcpkg remove --outdated
  - vcpkg install libmysql
  - vcpkg integrate install

vcpkg: librerías a instalar IV
vcpkg install nombre_libreria

- Librerías **ORM**
  - (***En estas pongo el comando completo que hay que lanzar***)
  - **vcpkg install soci** → Con esto no se instala un backend de mysql, faltaría:
  - **vcpkg install soci[core,mysql]**

- Base de datos cliente de postgresql → **libpqxx**
- Base de datos NoSQL, clave-valor (redis) → **hiredis**

vcpkg: librerías a instalar V
vcpkg install nombre_libreria

- Json Web Tokens → **jwt-cpp**
- Cifrado, certificados, seguridad → **openssl**

vcpkg: Liberar espacio

- Se pueden comprobar las librerías instaladas con:
- **vcpkg list**

- **Liberar espacio en vcpkg (cuando se han instalado las librerías)**
  - **Vaciar las carpetas:**
    - **buildtrees (todo el contenido)**
    - **packages (todo el contenido)**
    - **downloads (solo los archivos comprimidos, no borrar la carpeta tools)**

Otras herramientas

- **Postman**:
  - Para hacer pruebas con los servicios
  - Se puede utilizar la cuenta de Google con la verificación en dos pasos.
  - https://www.postman.com/downloads/

- **OpenSSL**
  - Descargar la última versión, **añadir al PATH**
  - Desde una consola podemos teclear el comando: **openssl** y comprobar que responde.
  - **No instalar la versión light**
  - https://slproweb.com/products/Win32OpenSSL.html

File	Type
Win64 OpenSSL v3.5.4 Light EXE | MSI	6MB Installer
Win64 OpenSSL v3.5.4 EXE | MSI	281MB Installer

Overview | DEL Post data | No environment

REST API basics: CRUD, test & variable / **Post data**

Save | Share

DELETE http://localhost:8080/items/1 **Send**

Params | Authorization | Headers (9) | Body | Scripts | Tests | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Beautify

```
1 {
2   "value": "Estudiar Boost.Beast"
3 }
4
```

Body | Cookies | Headers (3) | Test Results (1/1)

200 OK · 7 ms · 151 B | Save Response

{} JSON | Preview | Visualize

```
1 {
2     "id": 1,
3     "message": "Elemento eliminado",
4     "status": "ok"
5 }
```

PRUEBAS CON POSTMAN
Durante el curso

Bases de datos y otras herramientas

- **Utilizaremos contenedores de Docker**, esto se puede ir realizando durante el curso, suelen ser prácticas habituales con Docker.

- **Bases de datos:**
  - Mysql
  - Postgresql
  - Habrá backups de las BD en el curso

Instalación Docker Desktop en Windows

Antonio Espín Herranz

Instalación en Windows

- Contenidos
  - Docker desktop for Windows
  - Que es WLS2
  - Instalación de Docker Desktop

Docker Desktop for Windows

- **WSL2 → Windows SubSystem for Linux**
  - Permite mantener un S.O. de Linux dentro de Windows.
  - Nos permite utilizar los comandos de Linux
  - Desde el directorio donde arranca, podemos acceder a las unidades de Windows, haciendo:
    - cd /mnt → Se montan dentro de la carpeta mnt
    - Veremos todas las unidades de Windows como si fueran directorios.
    - ls -la

- **La mejor forma para instalar Docker en Windows**.
  - WSL2 es más ligero y eficiente que utilizar la máquina virtual dentro de Windows

Requisitos WSL2

- Requisitos Software
  - Windows 11 64-bit: Home o Pro version 21H2 o superior, Enterprise
    - Education version 21H2 o superior
  - Windows 10 64-bit: Home o Pro 2004 (build 19041) o superior, o Enterprise or Education 1909 (build 18363) o superior
  - Activar WSL 2 en Windows.
  - Descargue e instale el paquete de actualización del kernel de Linux ya lo hace automáticamente, no hay que descargar nada.

- Requisitos Hardware
  - Procesador de 64-bit con Second Level Address Translation (SLAT)
  - 4GB RAM
  - BIOS- Activar la virtualización

Instalación en Windows con WSL2

- Primero hay que instalar WSL2, comprobar que nuestro sistema cumple con los requisitos.
  - Mirar diapositivas anteriores.
  - ***Si tenemos Windows 11, ya lo lleva integrado y no es necesario instalar***.

- Otra forma de comprobar la versión de Windows y la compilación:
  - Abrir una consola de Windows: **CMD**
  - Y utilizar el comando: **winver**

Requisitos WSL2 – Comprobar Sistema

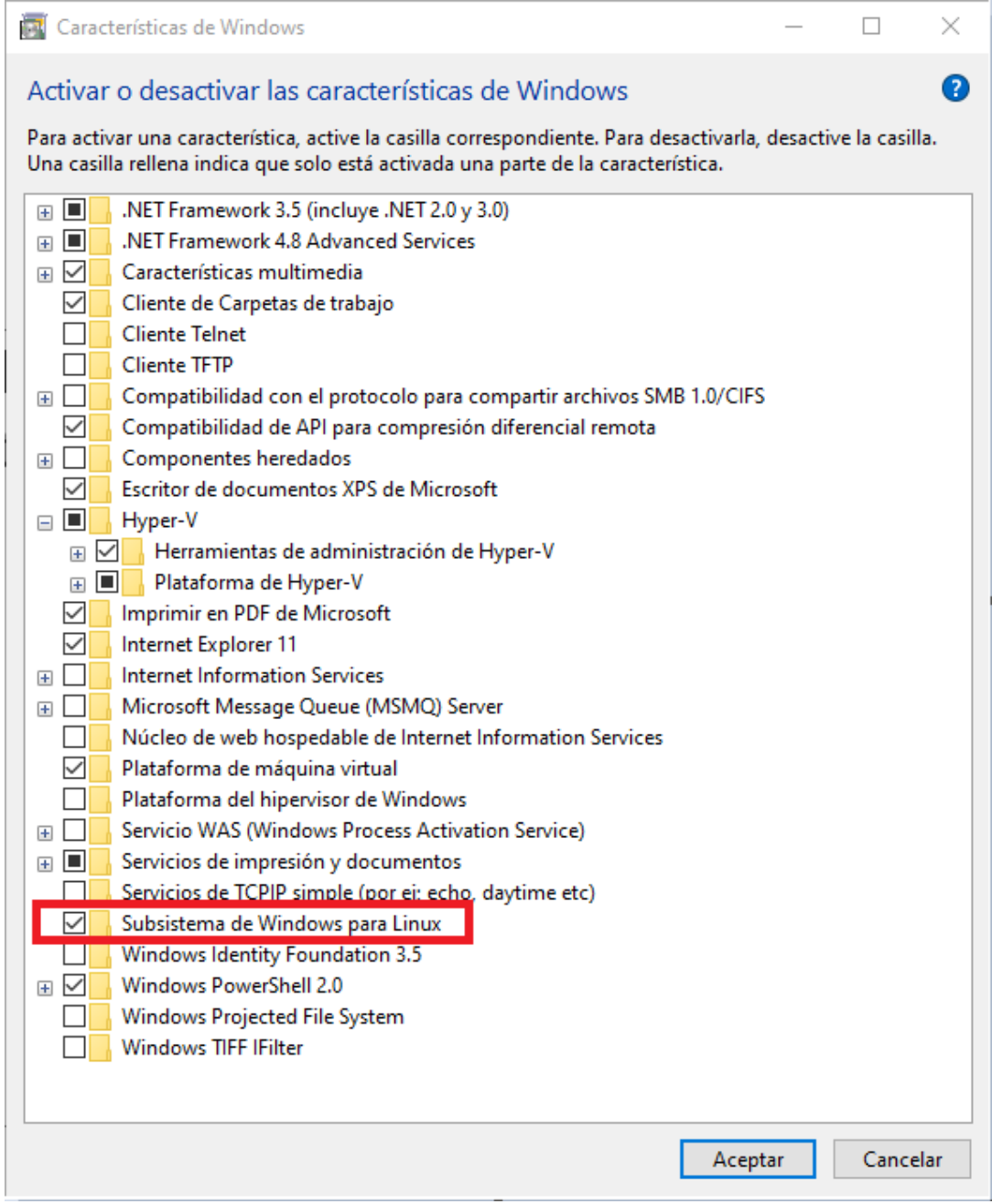
Especificaciones de Windows

Edición	Windows 10 Home
Versión	22H2
Instalado el	15/09/2020
Compilación del sistema operativo	19045.5487
Experiencia	Windows Feature Experience Pack 1000.19061.1000.0

Saber si lo tenemos instalado: WSL

- En la barra de búsqueda:
  - Teclear → activar o desactivar las características de Windows que no se añadieron en el momento de la instalación del S.O.

  - Si no está activado → **ACTIVARLO**
  - Realiza la búsqueda y hay que reiniciar.



Comprobar el comando: **wsl**

- Tenemos que entrar en una consola de Windows y teclear el comando: **wsl**

- Al principio no tendremos una distribución de Linux instalada y nos mostrará un mensaje:
  - Nos hace referencia a un enlace: https://aka.ms/wslstore
    - Desde esta página nos guía en el proceso de instalación y **actualización de WSL a WSL2**

- ***En caso tener ya instalada la distribución de Linux en el sistema nos mostrará que distribución y versión Linux tenemos instalada***.

Comprobar el comando: **wsl**

- Comprobar que distribuciones son válidas y cuáles se pueden instalar, comando → **wsl –l –o**

```
C:\Users\Anton>wsl -l -o
A continuación, se muestra una lista de las distribuciones válidas que se pueden instalar.
Instalar con 'wsl --install -d <Distribución>'

NAME                            FRIENDLY NAME
Ubuntu                          Ubuntu
Debian                          Debian GNU/Linux
kali-linux                      Kali Linux Rolling
Ubuntu-18.04                    Ubuntu 18.04 LTS
Ubuntu-20.04                    Ubuntu 20.04 LTS
Ubuntu-22.04                    Ubuntu 22.04 LTS
Ubuntu-24.04                    Ubuntu 24.04 LTS
OracleLinux_7_9                 Oracle Linux 7.9
OracleLinux_8_7                 Oracle Linux 8.7
OracleLinux_9_1                 Oracle Linux 9.1
openSUSE-Leap-15.6              openSUSE Leap 15.6
SUSE-Linux-Enterprise-15-SP5    SUSE Linux Enterprise 15 SP5
SUSE-Linux-Enterprise-15-SP6    SUSE Linux Enterprise 15 SP6
openSUSE-Tumbleweed             openSUSE Tumbleweed
```

Instalar la distribución de Linux

- Por ejemplo: con Ubuntu
  - **wsl --install –d Ubuntu**
  - Tiene que ser una de las que aparezca en la lista. Al poner el nombre de la distribución se tomará por defecto la más moderna. Ahora Ubuntu-24.04

  - **Descarga e instala la distribución elegida.**
  - Una vez que esté instalado podemos trabajar con Ubuntu.
  - Nos pedirá crear un usuario de Linux (utilizar el mismo que en Windows)

  - Es un Ubuntu nativo y funciona como en una máquina Ubuntu

Comprobaciones después de instalar

```
C:\Users\Anton>wsl -l -v
  NAME      STATE           VERSION
* Ubuntu    Running         1

C:\Users\Anton>wsl --list
Distribuciones del subsistema de Windows para Linux:
Ubuntu (predet.)

C:\Users\Anton>wsl
antonio@DESKTOP-6FPUMPM:/mnt/c/Users/Anton$
```

Del último comando salimos con **exit**

Para ver los comandos disponibles → wsl --help

Comprobaciones después de instalar

- Si queremos acceder a los ficheros de Windows desde la parte de Ubuntu:
  - Desde una consola de Windows: **wsl**
    - **Por defecto nos sitúa en el directorio home, por ejemplo: /home/antonio**
  - Buscar el programa: Ubuntu en Windows (**abrirá una consola directamente para trabajar con Ubuntu**)
  - Si nos movemos a la carpeta: **/mnt** tenemos acceso a las unidades de Windows, la tratamos como directorios y podemos movernos por los directorios de Windows.

Activar wsl 2

- Docker Desktop necesita tener activado WSL 2
- Podemos hacer:
  - wsl –l –v
  - wsl –set-version Ubuntu 2

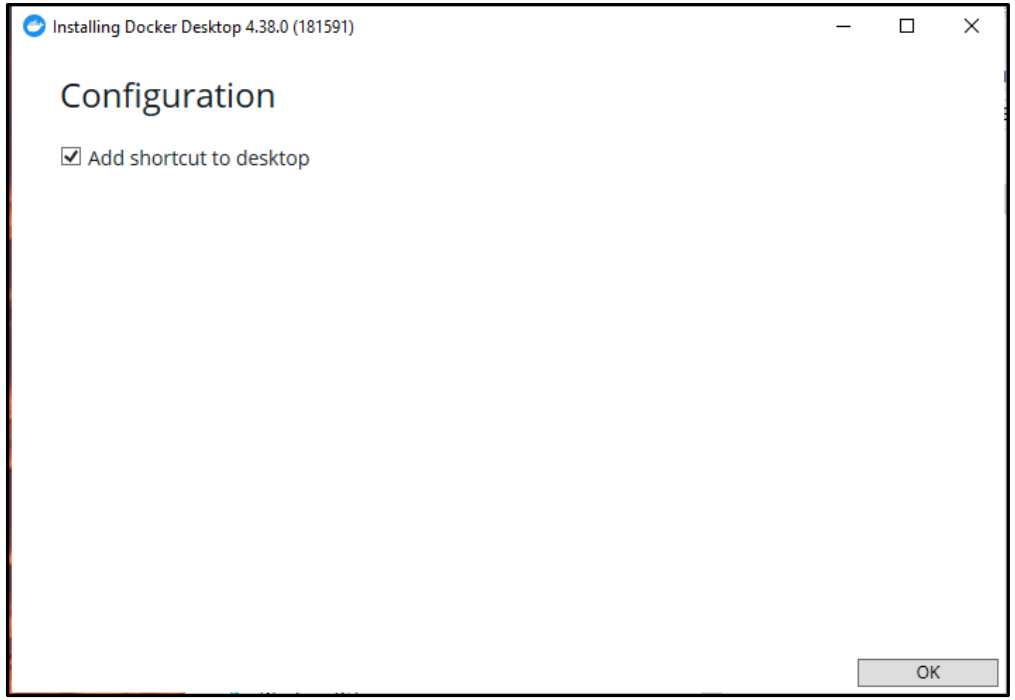
```
C:\Users\Anton>wsl -l -v
  NAME      STATE           VERSION
* Ubuntu    Running         1

C:\Users\Anton>wsl --set-version Ubuntu 2
Conversión en curso, esto puede tardar unos minutos...
Para información sobre las diferencias clave con WSL 2, visita https://aka.ms/wsl2
Conversión completada.

C:\Users\Anton>wsl -l -v
  NAME      STATE           VERSION
* Ubuntu    Stopped         2
```

Instalar Docker Desktop

- Documentación oficial (con los requisitos)
  - **Tener ya instalado y activado WSL 2**
  - https://docs.docker.com/desktop/setup/install/windows-install/

Instalar Docker Desktop

- Nos hará falta una cuenta en Docker, cuando entremos en la aplicación.
- Desempaqueta e instala todas las librerías y archivos necesarios.
- Cuando termina puede pedir un reinicio.
  - Si no está activado WSL2 nos preguntará que si lo queremos activar.
- Al arrancar Docker desktop hay que aceptar los términos de uso.



Posibles errores al arrancar

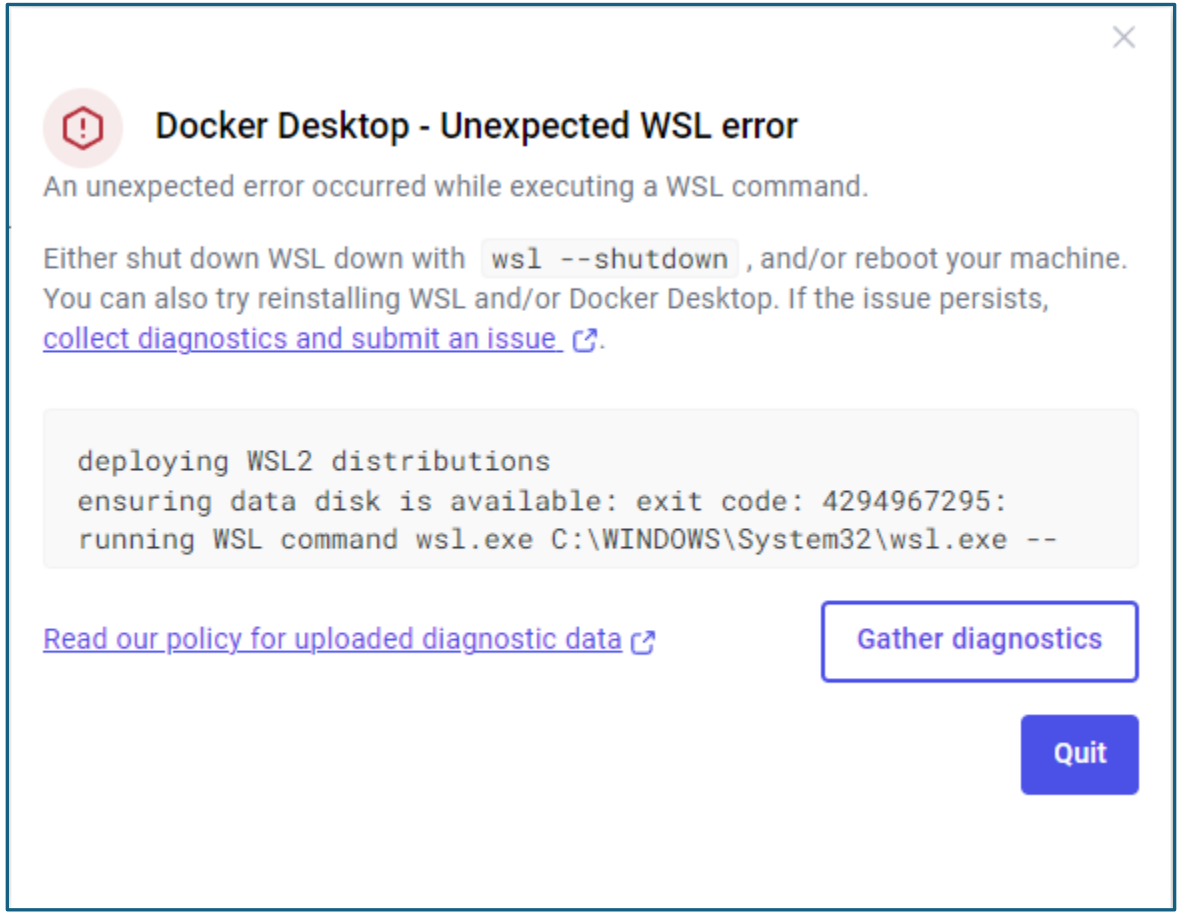
- https://forums.docker.com/t/an-unexpected-error-was-encountered-while-executing-a-wsl-command/137525

- https://stackoverflow.com/questions/76160943/docker-desktop-an-unexpected-error-was-encountered-while-executing-a-wsl-comma

- Comandos:
  - **wsl –l –v**

- ***En un caso función lanzar el comando wsl (ver los dos arrancados)***

```
C:\Users\Anton>wsl -l -v
  NAME              STATE           VERSION
* Ubuntu            Running         2
  docker-desktop    Running         2
```



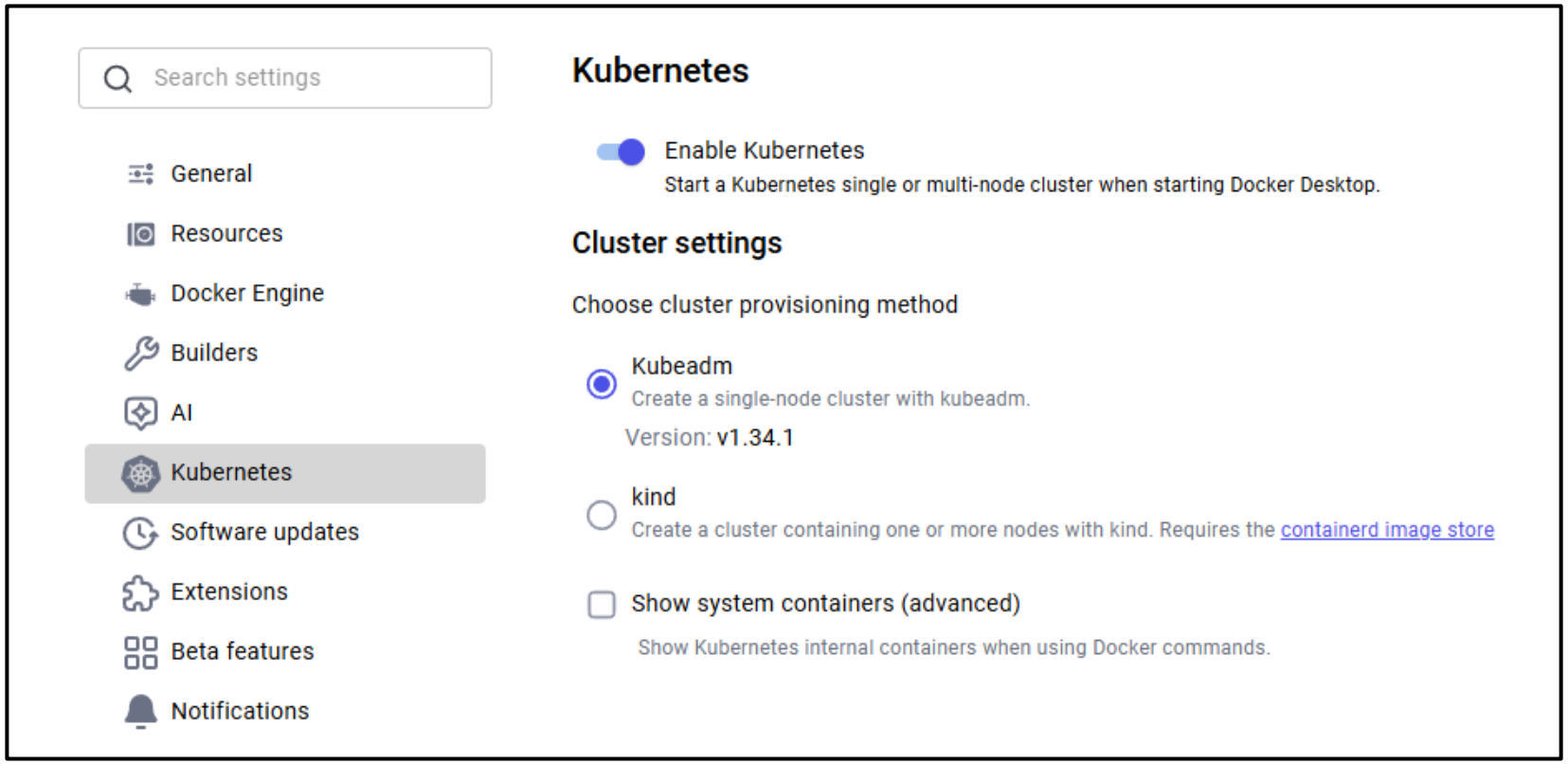
Instalación

- **Kubernetes** se puede **instalar** a partir de **Docker Desktop**.
- Ya viene preparado para instalar:
  - Ir a **settings**

  - Activar la opción de **kubernetes**:

OJO, la activación de Kubernetes puede llevar un tiempo.

Instalación



- Activar "**Enable Kubernetes**"
- Hay que pulsar el botón
  - "**Apply**"
- También hay que probar desde una consola el comando **kubectl** (es equivalente al comando Docker)
- Con este comando se envían instrucciones al Clúster de Kubernetes.

El comando **kubectl** suele estar disponible Antes de activar kubernetes.

```
C:\Users\Anton>kubectl version
Client Version: v1.34.1
Kustomize Version: v5.7.1
Unable to connect to the server: dial tcp [::1]:8080: connectex:
```

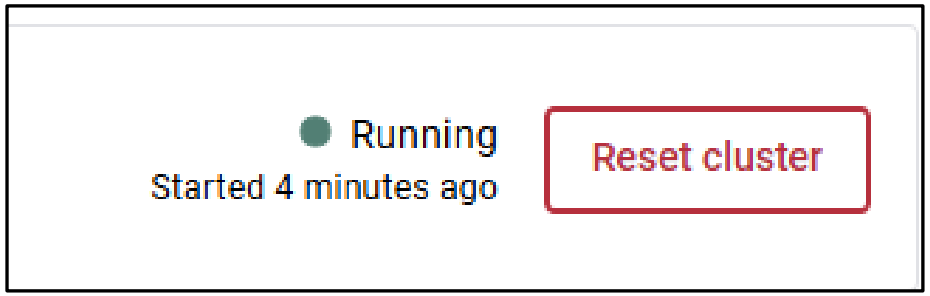
El comando responderá antes de activar k8s, pero no está activado.

K8s activado

- Una vez activemos k8s el mismo comando de antes:

```
C:\Users\Anton>kubectl version
Client Version: v1.34.1
Kustomize Version: v5.7.1
Server Version: v1.34.1
```

- Y también en Docker desktop veremos:



minikube

- **Minikube** es una herramienta diseñada para **ejecutar Kubernetes localmente en tu máquina**.

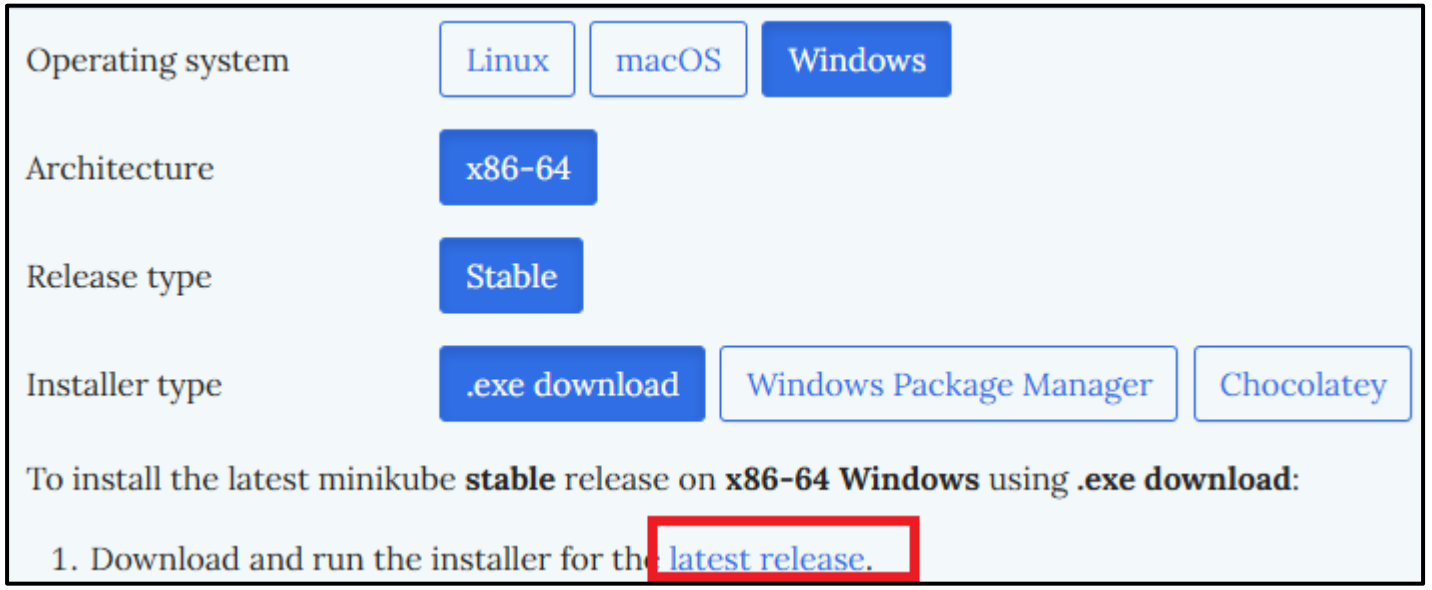
- **Kubernetes** es una **plataforma de orquestación de contenedores** que permite desplegar, gestionar y escalar aplicaciones en contenedores.

- **Minikube**, permite crear un entorno Kubernetes en tu ordenador para **pruebas**, **desarrollo** o **aprendizaje** sin necesidad de configurar un clúster completo.

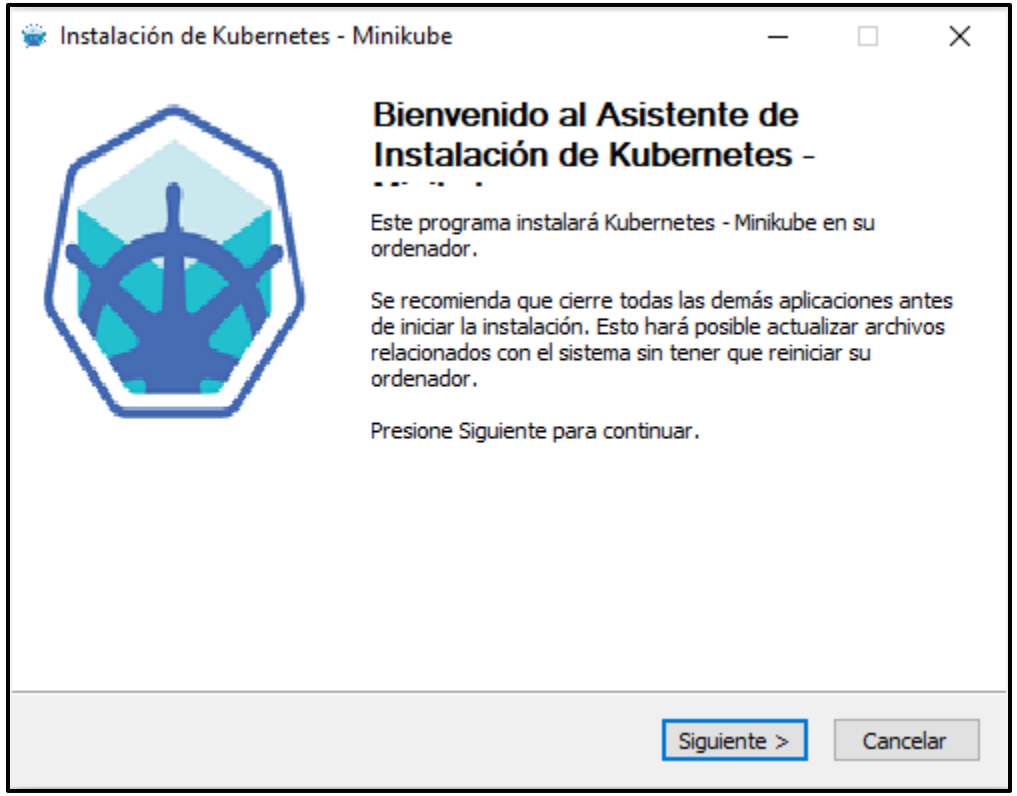
Instalar minikube

- **Enlace a la instalación de minikube:**
  - https://minikube.sigs.k8s.io/docs/start/?arch=%2Fwindows%2Fx86-64%2Fstable%2F.exe+download

  - **Selecciona los parámetros de la instalación.**
    - Sistema Operativo
    - Arquitectura
    - Release
    - Tipo de instalación.



Instalar minikube



- Seleccionar idioma y siguiente
- Aceptar la licencia.
- Seleccionar la ruta de instalación.
- Ya lo añade al **PATH**

Iniciar minikube

- Probar el comando desde una **consola**:

```
C:\Users\Anton>minikube version
minikube version: v1.37.0
commit: 65318f4cfff9c12cc87ec9eb8f4cdd57b25047f3
```

- Lanzar el comando: **minikube start**
  - Minikube necesita un **driver**, si tenemos Docker instalado es una opción que podemos utilizar.

- Al lanzar el comando minikube start selecciona automáticamente el driver de Docker.

minikube start (salida del comando)

```
C:\Users\Anton>minikube start
* minikube v1.37.0 en Microsoft Windows 10 Home 10.0.19045.6332 Build 19045.6332
* Controlador docker seleccionado automáticamente
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.48 ...
* Descargando Kubernetes v1.34.0 ...
    > gcr.io/k8s-minikube/kicbase...:  488.52 MiB / 488.52 MiB  100.00% 54.79 M
    > preloaded-images-k8s-v18-v1...:  337.07 MiB / 337.07 MiB  100.00% 15.95 M
* Creating docker container (CPUs=2, Memory=8100MB) ...
! Failing to connect to https://registry.k8s.io/ from both inside the minikube container and host machine
* To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
* Preparando Kubernetes v1.34.0 en Docker 28.4.0...
* Configurando CNI bridge CNI ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Complementos habilitados: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

Deberíamos de ver una salida similar a esta.
Se crea un contenedor que actúa de control de kubernetes.

minikube

- Si hay algún problema con la instalación:
  - **minikube delete**

```
C:\Users\Anton>minikube delete
* Eliminando "minikube" en docker...
* Eliminando contenedor "minikube" ...
* Eliminando C:\Users\Anton\.minikube\machines\minikube...
* Removed all traces of the "minikube" cluster.
```

  - **minikube start --driver=docker**

Interactúa con el cluster

- Se pueden lanzar de prueba estos comandos:
  - **kubectl get po –A**
    - Listar todos los **pods** en **todos los namespaces** del clúster.
  - **minikube dashboard**
    - Arranca un navegador con el clúster de kubernetes.

Servicio (***no hace falta probar esto***)

Cree una implementación de muestra y expóngala en el puerto 8080:

```
kubectl create deployment hello-minikube --image=kicbase/echo-server:1.0
kubectl expose deployment hello-minikube --type=NodePort --port=8080
```

Puede tomar un momento, pero su implementación aparecerá pronto cuando ejecute:

```
kubectl get services hello-minikube
```

La forma más fácil de acceder a este servicio es dejar que minikube inicie un navegador web para usted:

```
minikube service hello-minikube
```

Alternativamente, use kubectl para reenviar el puerto:

```
kubectl port-forward service/hello-minikube 7080:8080
```

¡Listo! Tu aplicación ya está disponible en http://localhost:7080/ .

Deberías poder ver los metadatos de la solicitud en la salida de la aplicación. Intenta cambiar la ruta de la solicitud y observa los cambios. De igual forma, puedes ejecutar una solicitud POST y observar cómo aparece el cuerpo en la salida.